

EBOOK DEVOPS

Docker & Tự động hóa đóng gói

Git · CI/CD · Docker · Docker Compose · Docker Swarm

Biên soạn cho khóa học 8 giờ · 29 tháng 7, 2026

Mục lục

PHẦN I — NỘI DUNG

0 · Tư duy DevOps	4
1 · Version Control	7
2 · CI/CD	12
3 · Docker Fundamentals	17
4 · Docker Compose	21
5 · Docker Swarm	27

PHẦN II — BÀI GIẢI

1 · Git & GitLab	33
2 · Pipeline CI/CD	36
3 · 5 lab Docker	40
4 · Compose & deploy	43
5 · Docker Swarm	47

PHỤ LỤC — THAM KHẢO

Reference — Sách & Video	51
--------------------------------	----

PHẦN I

Nội dung

NỘI DUNG · 15'

Tư duy DevOps: tại sao nó tồn tại?

Trước khi học công cụ, hãy hiểu vấn đề mà DevOps sinh ra để giải quyết — bức tường ngăn cách giữa Dev và Ops.

Hầu hết tài liệu DevOps bắt đầu bằng danh sách công cụ: Git, Jenkins, Docker, Kubernetes... Nhưng công cụ chỉ là *hệ quả*. Nếu không hiểu **vấn đề gốc**, bạn sẽ dùng công cụ sai chỗ và tự hỏi vì sao pipeline của mình phức tạp mà chẳng nhanh hơn.

Bài này trả lời một câu hỏi duy nhất: **DevOps sinh ra để giải quyết cái gì?**



Vấn đề gốc: “bức tường hoang mang” (wall of confusion)

Trong mô hình truyền thống, hai đội có **mục tiêu mâu thuẫn nhau**:

- **Dev (Development)** được thưởng vì *ra tính năng mới nhanh* → họ muốn **thay đổi liên tục**.
- **Ops (Operations)** được thưởng vì *hệ thống ổn định, không sập* → họ muốn **hạn chế thay đổi**.

Thay đổi chính là nguồn gốc của cả tính năng mới *lẫn* sự cố. Vì vậy hai đội kéo về hai hướng ngược nhau. Kết quả kinh điển:

Dev quăng code qua tường cho Ops kèm câu “chạy tốt trên máy tôi mà”. Ops nhận một thứ họ không hiểu, deploy lên production, nó sập, và không ai chịu trách nhiệm.

Tại sao đây là vấn đề đắt đỏ? Vì mỗi lần “quăng qua tường” là một lần mất thông tin: Dev biết code hoạt động thế nào, Ops biết production trông ra sao, nhưng không bên nào có bức tranh đầy đủ. Sự cố xảy ra ở đúng khoảng trống đó.

DevOps là câu trả lời: xóa bức tường

DevOps **không phải một chức danh** và cũng **không phải một công cụ**. Nó là một cách tổ chức công việc để Dev và Ops chia sẻ *cùng một mục tiêu* và *cùng một trách nhiệm*: “you build it, you run it”.

Ba trụ cột:

1. **Văn hóa (Culture)** — cùng chịu trách nhiệm cho toàn bộ vòng đời sản phẩm, không đổ lỗi.
2. **Quy trình (Process)** — tự động hóa những bước lặp lại và dễ sai (build, test, deploy).
3. **Công cụ (Tools)** — Git, CI/CD, Docker... chỉ là *phương tiện* để thực thi hai điều trên.

Nếu chỉ mua công cụ mà không đổi văn hóa, bạn có "Jenkins nhưng vẫn quăng qua tường" — đó là lý do nhiều tổ chức "làm DevOps" mà không thấy hiệu quả.

Vòng đời DevOps: tại sao là vòng lặp vô tận?

DevOps thường được vẽ bằng biểu tượng số 8 nằm ngang (∞). Đó không phải để cho đẹp — nó nói lên rằng phần mềm **không bao giờ "xong"**: mỗi lần vận hành sinh ra dữ liệu để lập kế hoạch cho vòng tiếp theo.

Giai đoạn	Câu hỏi trả lời	Công cụ trong khóa
Plan	Làm gì tiếp theo?	Issue, board
Code	Viết & quản lý mã nguồn thế nào?	Git, GitLab, GitHub
Build	Biến code thành sản phẩm chạy được?	CI, Docker build
Test	Có chắc nó đúng không?	CI pipeline
Release	Đưa ra phiên bản nào?	CD, Registry
Deploy	Đẩy lên môi trường thật?	Docker, Swarm
Operate	Giữ nó chạy?	Swarm, orchestration
Monitor	Nó có khỏe không?	Logs, metrics

Tại sao rút ngắn vòng lặp lại quan trọng? Vòng lặp càng ngắn, phản hồi càng nhanh, và mỗi lần thay đổi càng nhỏ. Thay đổi nhỏ = ít rủi ro, dễ tìm nguyên nhân khi lỗi, dễ rollback. Đây là lý do sâu xa vì sao ta muốn *Continuous Integration* và *Continuous Deployment*: không phải để "deploy cho nhiều", mà để mỗi lần deploy trở nên nhỏ và nhàm chán — nhàm chán là tốt trong vận hành.

Bốn khái niệm bạn sẽ gặp lại suốt khóa

- **CI — Continuous Integration:** mỗi commit được tự động build + test ngay, thay vì gộp code hàng tuần rồi "integration hell". *Vì sao?* Lỗi tích hợp phát hiện sau 5 phút rẻ hơn nhiều so với sau 5 ngày.
- **CD — Continuous Delivery/Deployment:** sau khi qua test, phần mềm luôn ở trạng thái *sẵn sàng phát hành* (Delivery) hoặc *tự động phát hành* (Deployment).
- **laC — Infrastructure as Code:** mô tả hạ tầng bằng file (Dockerfile, docker-compose.yml) thay vì bấm tay. *Vì sao?* File thì version được, review được, tái tạo được — "máy tôi" và "production" cuối cùng giống nhau.
- **GitOps:** Git là *nguồn chân lý duy nhất* — muốn đổi production thì sửa file trong Git, không SSH vào server gõ tay.

Bức tranh toàn khóa

Toàn bộ khóa học này là hành trình đi dọc vòng lặp đó, ráp các mảnh lại thành một pipeline hoàn chỉnh:

```
Code   →   Build/Test   →   Package   →   Ship   →   Run
Git     GitLab CI /      Docker       Registry  Docker
GitLab  GitHub Actions  image           Swarm
```

Đến cuối khóa, khi bạn `git push`, một chuỗi tự động sẽ chạy: CI build & test → đóng gói thành Docker image → đẩy lên registry → deploy lên cụm Swarm. Bạn sẽ hiểu *vì sao* cần từng mắt xích, không chỉ *cách* gõ lệnh.

Đầu ra của bài: Bạn vẽ được sơ đồ `commit → build → image → deploy` và giải thích được vì sao mỗi mũi tên tồn tại.

Tiếp theo: **Version Control** — vì sao Git thăng thế →

NỘI DUNG · 75'

Version Control: Git, GitHub & GitLab

Vì sao mọi đội phần mềm đều cần VCS, vì sao Git là hệ phân tán, và GitHub khác GitLab ở đâu.

Version Control System (VCS) là nền móng của mọi thứ còn lại trong DevOps. Nếu không kiểm soát được *code thay đổi thế nào*, bạn không thể tự động hóa build, test hay deploy một cách đáng tin cậy.



Tại sao cần Version Control?

Hãy tưởng tượng làm việc *không có* VCS. Bạn sẽ có `baocao_final.doc`, `baocao_final_v2.doc`, `baocao_final_thatsu.doc`. Với code và nhiều người cùng sửa, thảm họa này nhân lên gấp bội. VCS giải quyết bốn nhu cầu cốt lõi:

1. **Lịch sử (History):** ai đổi gì, khi nào, *vì sao* — mỗi commit là một ảnh chụp có thể quay lại.
2. **Phối hợp (Collaboration):** nhiều người sửa cùng lúc mà không đè lên nhau.
3. **Nhánh (Branching):** thử nghiệm tính năng mới mà không phá bản đang chạy.
4. **Khôi phục (Rollback):** production hỏng → quay về commit lành trong vài giây.

Tại sao "vì sao" của một thay đổi lại quan trọng hơn "cái gì"? Diff cho bạn biết dòng nào đổi, nhưng 6 tháng sau, thứ bạn cần là *lý do* đổi. Đó là lý do commit message tốt (giải thích tại sao) đáng giá hơn commit message kiểu "fix bug". Git lưu cả hai, nhưng chỉ con người viết được phần "tại sao".

Tại sao Git là hệ phân tán (distributed)?

Trước Git có SVN — hệ **tập trung (centralized)**: chỉ có *một* kho trung tâm, muốn commit hay xem lịch sử đều phải kết nối tới server. Server sập = cả đội tê liệt.

Git **phân tán**: mỗi máy có *bản sao đầy đủ* của toàn bộ lịch sử. Điều này dẫn tới những hệ quả rất thực tế:

- **Làm việc offline:** commit, xem log, tạo nhánh — tất cả cục bộ, không cần mạng.
- **Không có điểm chết duy nhất:** mọi clone đều là một bản backup đầy đủ.
- **Nhánh siêu rẻ:** tạo nhánh chỉ là tạo một con trỏ 40 ký tự → khuyến khích thử nghiệm.

Tại sao nhánh rẽ lại thay đổi cách làm việc? Khi tạo nhánh tốn kém (như SVN), người ta ngại tạo và gộp code khổng lồ mỗi lần → xung đột lớn. Khi nhánh rẽ, ta tạo nhánh cho *từng* tính năng nhỏ, merge sớm và thường xuyên — chính là tinh thần Continuous Integration ở bài trước.

Ba trạng thái của một file trong Git

Đây là mô hình khiến người mới bối rối nhất, nhưng hiểu nó thì mọi lệnh Git trở nên logic:

```
Working Directory  —git add→  Staging Area  —git commit→  Repository
(đang sửa)                (chuẩn bị lưu)                (đã lưu
vĩnh viễn)
```

- **Working Directory:** file thật bạn đang sửa.
- **Staging Area (Index):** khu vực "soạn" — bạn chọn *chính xác* thay đổi nào đi vào commit tới.
- **Repository:** lịch sử đã cam kết.

Tại sao cần Staging Area — sao không commit thẳng? Vì một buổi làm việc thường lẫn lộn nhiều thay đổi. Staging cho bạn tách chúng thành các commit sạch, mỗi commit một ý nghĩa. Commit sạch = lịch sử đọc được = rollback chính xác được.

Các lệnh Git thường dùng

```
git init                # khởi tạo repo mới
git clone <url>         # sao chép repo về máy
git status              # xem trạng thái file
git add <file>          # đưa thay đổi vào staging
git commit -m "message" # lưu snapshot kèm lý do
git push                # đẩy commit lên remote
git pull                # kéo commit mới nhất về (fetch + merge)
git branch feature/x    # tạo nhánh mới
git checkout feature/x  # chuyển sang nhánh (hoặc: git switch)
git merge feature/x     # gộp nhánh vào nhánh hiện tại
git log --oneline        # xem lịch sử ngắn gọn
git stash                # cất tạm thay đổi chưa commit
```

Nhánh và mô hình làm việc: chọn cái nào và vì sao

Mô hình	Ý tưởng	Hợp với
Git Flow	Nhiều nhánh dài hạn (<code>develop</code> , <code>release</code> , <code>hotfix</code>)	Sản phẩm có phiên bản phát hành rõ ràng
GitHub Flow	Chỉ <code>main</code> + nhánh feature ngắn, merge qua Pull Request	Web app deploy liên tục
Trunk-based	Mọi người commit vào <code>main</code> , nhánh cực ngắn	Đội trưởng thành, CI mạnh

Tại sao xu hướng dịch về nhánh ngắn (GitHub Flow / Trunk-based)? Nhánh sống càng lâu, càng "trôi" xa `main` , merge càng đau. Nhánh ngắn ép ta tích hợp thường xuyên — lại đúng tinh thần CI. Git Flow không sai, nhưng nó sinh ra cho thời kỳ phát hành theo quý, không phải deploy nhiều lần mỗi ngày.

GitHub vs GitLab: khác nhau ở đâu?

Cả hai đều là nền tảng *host* repo Git kèm công cụ cộng tác. Điểm khác biệt nằm ở triết lý:

Tiêu chí	GitHub	GitLab
CI/CD	GitHub Actions (marketplace lớn)	GitLab CI/CD tích hợp sẵn
Self-hosted	GitHub Enterprise (trả phí)	GitLab CE miễn phí, tự host
Container Registry	Có (GHCR)	Tích hợp sẵn trong repo
Triết lý	Trung tâm của open-source thế giới	Nền tảng DevOps all-in-one

Vì sao khóa này dùng cả hai? GitLab cho bạn trải nghiệm "một nền tảng lo trọn vòng đời" và khả năng tự host runner — quý cho môi trường doanh nghiệp/on-premise. GitHub cho bạn hệ sinh thái Actions khổng lồ và là nơi phần lớn open-source sống. Hiểu cả hai giúp bạn không bị khóa vào một nhà cung cấp.

Tự cài GitLab (self-hosted) — vì sao và khi nào

Với `gitlab.com` bạn có mọi thứ sẵn. Nhưng doanh nghiệp thường cần **self-host** vì lý do bảo mật/tuân thủ: code không được rời khỏi mạng nội bộ.

Cài GitLab CE bằng gói Omnibus trên Ubuntu 22.04:

```
# Cài phụ thuộc
sudo apt update
sudo apt install -y curl openssh-server ca-certificates tzdata perl

# Thêm repo GitLab và cài
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.deb.sh | sudo bash
sudo EXTERNAL_URL="http://gitlab.example.com" apt install gitlab-ce

# Sau khi sửa cấu hình, áp dụng lại
sudo gitlab-ctl reconfigure
```

Tại sao `external_url` quan trọng? GitLab sinh ra link clone, link CI, webhook... dựa trên URL này. Đặt sai thì runner và người dùng nhận địa chỉ không truy cập được. Đây là lỗi cấu hình phổ biến nhất khi tự host.

Thực hành (xem đáp án ở mục Bài giải)

1. Tạo repo trên GitLab, `clone` về máy.
2. Tạo nhánh `feature/login`, commit thay đổi, `push`.
3. Tạo **Merge Request**, review, merge vào `main`.
4. Cố tình tạo xung đột giữa hai nhánh và giải quyết.

Đầu ra của bài: Bạn giải thích được vì sao Git phân tán, phân biệt được 3 trạng thái file, và chọn được mô hình nhánh phù hợp với đội mình.

Tiếp theo: [CI/CD — biến mỗi commit thành một sản phẩm](#) → · **Bài giải:** [Git & GitLab](#)

NỘI DUNG · 90'

CI/CD: GitLab Runner & GitHub Actions

Vì sao tự động hóa build/test lại thay đổi cuộc chơi, và cách một pipeline thực sự vận hành.

Ở bài đầu ta nói CI/CD giúp “rút ngắn vòng lặp phản hồi”. Bài này đi sâu vào *vì sao* điều đó tạo ra khác biệt lớn đến vậy, và *cơ chế* thực sự đằng sau một pipeline.



Tại sao CI/CD tồn tại: nỗi đau “integration hell”

Trước CI, các đội làm việc trên nhánh riêng hàng tuần rồi mới gộp. Đến ngày merge, hàng nghìn dòng va nhau — người ta gọi đó là **integration hell**. Chi phí sửa một lỗi tăng theo cấp số nhân với thời gian nó tồn tại chưa bị phát hiện.

Continuous Integration đảo ngược điều này: *mỗi commit* được tự động build và test ngay. Lỗi lộ ra sau vài phút, khi thay đổi còn nhỏ và bạn còn nhớ mình vừa làm gì.

Tại sao “tự động” mới là chìa khóa, không phải “test”? Đội nào cũng có test. Vấn đề là con người quên chạy, chạy trên máy khác cấu hình, hoặc bỏ qua khi vội. Máy thì không quên và không thiên vị. Tự động hóa biến “nên làm” thành “luôn xảy ra” — đó mới là giá trị thật.

CI vs CD vs CD (Delivery vs Deployment)

- **CI — Continuous Integration:** tự động **build + test** mỗi thay đổi.
- **CD — Continuous Delivery:** sau khi qua test, sản phẩm luôn *sẵn sàng phát hành*; bấm một nút là ra production.
- **CD — Continuous Deployment:** đi xa hơn — qua test là *tự động* lên production, không cần bấm nút.

Sự khác biệt giữa Delivery và Deployment chỉ là “có một nút bấm thủ công của con người hay không” — và đó là quyết định về *mức độ tin tưởng* vào bộ test của bạn.

Giải phẫu một pipeline

Bốn khái niệm cốt lõi, giống nhau ở cả GitLab lẫn GitHub:

Khái niệm	Là gì	Vì sao cần
Stage	Nhóm các job theo giai đoạn (build → test → deploy)	Chạy tuần tự, chặn giai đoạn sau nếu trước hỏng
Job	Một đơn vị công việc chạy script	Đơn vị nhỏ nhất để song song hóa & báo lỗi
Runner	Máy/agent thực thi job	Tách "định nghĩa việc" khỏi "nơi chạy việc"
Artifact	Kết quả job để lại (file build, báo cáo)	Truyền dữ liệu giữa các stage / lưu lại

Tại sao tách Runner khỏi định nghĩa pipeline? Vì "việc cần làm" (build, test) không nên gắn cứng với "máy nào làm". Hôm nay chạy trên runner Docker, mai chạy trên Kubernetes, một chạy trên máy có GPU — bạn chỉ đổi runner, không đổi `.gitlab-ci.yml`. Đây là nguyên tắc tách bạch (separation of concerns) kinh điển.

GitLab Runner: các executor và vì sao chọn Docker

Runner có nhiều **executor** — cách nó thực thi job:

- **shell:** chạy thẳng trên máy runner. Đơn giản nhưng "bẩn" — các job dùng chung môi trường, dễ dính rác của nhau.
- **docker:** mỗi job chạy trong một container *sạch* mới toanh. Đây là lựa chọn mặc định nên dùng.
- **kubernetes:** mỗi job là một pod — scale tự động theo tải.

Tại sao executor Docker gần như luôn là lựa chọn đúng? Vì nó cho bạn *môi trường sạch và tái lập* cho mỗi lần chạy. "Chạy được trên CI hôm qua, hôm nay hỏng" thường do rác tích tụ trên máy shell. Container mới mỗi lần = loại bỏ nguyên nhân đó tận gốc — cùng triết lý với Docker mà ta học ở bài sau.

File `.gitlab-ci.yml`

```

stages: [build, test, deploy]

build-job:
  stage: build
  image: node:20                # job chạy trong container node:20 sạch
  script:
    - npm ci
    - npm run build
  artifacts:
    paths: [dist/]              # giữ lại kết quả build cho stage sau

test-job:
  stage: test
  image: node:20
  script:
    - npm test

deploy-job:
  stage: deploy
  script:
    - echo "Deploying ..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"' # chỉ deploy khi ở nhánh main

```

Tại sao dùng `npm ci` chứ không `npm install` trong CI? `npm ci` cài chính xác theo `package-lock.json`, không cập nhật lock, và xóa `node_modules` cũ trước. Kết quả: build tái lập 100% giữa các lần chạy. Trong CI, tính tái lập quan trọng hơn sự tiện lợi.

GitHub Actions: cùng ý tưởng, cú pháp khác

GitHub Actions dùng cùng khái niệm nhưng gọi tên khác: **workflow** chứa **jobs**, mỗi job có **steps**, và có thể tái dùng **actions** từ marketplace.

```
# .github/workflows/build.yml
name: Build
on: [push] # kích hoạt khi có push

jobs:
  build:
    runs-on: ubuntu-latest # GitHub cấp máy ảo sạch
    steps:
      - uses: actions/checkout@v4 # tái dùng action lấy code
      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .
```

Tại sao gắn tag image bằng `${{ github.sha }}` ? Mỗi commit có một SHA duy nhất. Gắn image theo SHA nghĩa là mỗi image truy ngược được về đúng dòng code sinh ra nó — khi production lỗi, bạn biết chính xác image nào, commit nào. Gắn tag `latest` thì không truy vết được — đó là lý do `latest` nguy hiểm trong production.

GitLab CI vs GitHub Actions

	GitLab CI	GitHub Actions
File	<code>.gitlab-ci.yml</code> (gốc repo)	<code>.github/workflows/*.yml</code>
Máy chạy	Runner tự host hoặc shared	Runner GitHub hoặc self-hosted
Tái dùng	<code>include</code> , templates	Marketplace actions (<code>uses:</code>)
Mạnh ở	Tự host, kiểm soát toàn bộ	Hệ sinh thái action khổng lồ

Thực hành (xem đáp án ở mục Bài giải)

1. Register một GitLab Runner (executor Docker).
2. Viết `.gitlab-ci.yml` để build + test tự chạy khi push.
3. Xem pipeline chạy, đọc log, tải artifact.
4. Tạo GitHub Actions build Docker image cho cùng project.
5. So sánh trải nghiệm hai nền tảng.

Đầu ra của bài: Bạn giải thích được vì sao runner tách khỏi pipeline, vì sao container sạch mỗi lần chạy, và tự viết được một pipeline build-test-deploy.

Tiếp theo: [Docker — vì sao container thay thế “máy tôi”](#) → · **Bài giải:** [Dựng pipeline CI/CD](#)

NỘI DUNG · 75'

Docker Fundamentals

Vì sao container ra đời, khác máy ảo ở đâu, và ý nghĩa thật của Image, Container, Network, Volume, Registry.

Docker là mảnh ghép biến "chạy tốt trên máy tôi" thành "chạy giống nhau ở mọi nơi". Nhưng để dùng Docker đúng, bạn phải hiểu *vấn đề* nó giải quyết, không chỉ học thuộc lệnh `docker run`.



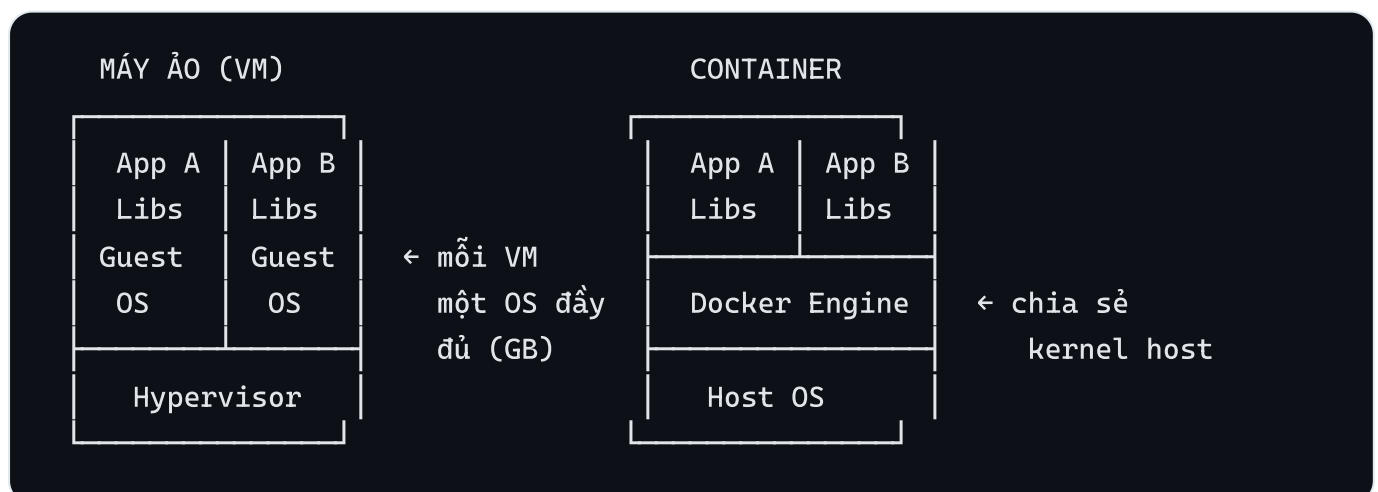
Tại sao Docker ra đời: vấn đề "môi trường không đồng nhất"

Một ứng dụng cần rất nhiều thứ để chạy: đúng phiên bản ngôn ngữ, đúng thư viện hệ thống, đúng biến môi trường. Máy dev, máy CI và server production luôn khác nhau ở đâu đó → sinh ra lỗi "chỗ tôi chạy được".

Docker đóng gói **ứng dụng + toàn bộ phụ thuộc** vào một *image* duy nhất. Image đó chạy y hệt ở mọi nơi có Docker. Vấn đề môi trường không đồng nhất biến mất.

Tại sao đây là điều CI/CD ở bài trước cần? Pipeline chỉ đáng tin nếu "sản phẩm build ra" giống nhau mọi lần. Nếu artifact là mã nguồn, môi trường chạy vẫn khác nhau. Nếu artifact là *Docker image*, môi trường được đóng băng bên trong. Docker chính là mắt xích "Package" trong vòng đời DevOps.

Container khác Máy ảo (VM) thế nào — và vì sao nhẹ hơn?



- **VM** ảo hóa cả *phần cứng*: mỗi VM chạy một hệ điều hành khách đầy đủ → nặng hàng GB, khởi động hàng phút.
- **Container** ảo hóa ở tầng *hệ điều hành*: chia sẻ kernel của host, chỉ đóng gói app + thư viện → nhẹ hàng chục MB, khởi động trong mili-giây.

Tại sao "nhẹ và nhanh" lại quan trọng đến thế? Vì nó thay đổi những gì bạn có thể làm.

Container khởi động trong tích tắc nên ta có thể tạo/hủy liên tục — chạy một container sạch cho mỗi job CI (bài trước), scale từ 3 lên 50 bản sao trong vài giây (Swarm, bài cuối). "Nhẹ" không phải chuyện tiết kiệm RAM, mà là mở ra cả một cách vận hành mới.

Năm khái niệm cốt lõi

1. Image — bản thiết kế bất biến

Image là *template read-only*, gồm nhiều **layer** xếp chồng. Mỗi lệnh trong Dockerfile tạo một layer. Image là thứ bạn *build một lần* rồi dùng lại khắp nơi.

2. Container — thực thể đang chạy

Container là một *instance đang chạy* của image. Cùng một image có thể sinh ra hàng trăm container. Quan hệ Image ↔ Container giống Class ↔ Object trong lập trình.

3. Network — cách container nói chuyện

Docker tạo mạng ảo để các container giao tiếp. Các loại: **bridge** (mặc định, mạng nội bộ trên một host), **host** (dùng chung mạng host), **overlay** (mạng trải nhiều host — dùng trong Swarm).

Tại sao container gọi nhau bằng tên chứ không bằng IP? Vì container sinh-diệt liên tục, IP thay đổi mỗi lần. Docker có DNS nội bộ: trong cùng một network, container **frontend** gọi **http://backend:8000** và Docker tự phân giải. Đây là lý do ta luôn đặt các service liên quan vào cùng một custom network.

4. Volume — dữ liệu sống lâu hơn container

Container vốn *phù du (ephemeral)* — xóa đi là mất sạch dữ liệu bên trong. Volume là vùng lưu trữ *tách khỏi* vòng đời container, do Docker quản lý.

Tại sao container cố tình "phù du"? Vì nó ép ta tách *code* (bất biến, trong image) khỏi *dữ liệu* (biến đổi, trong volume). Nhờ tách bạch này, bạn có thể nâng cấp app bằng cách thay container mới mà dữ liệu vẫn nguyên trong volume. Đây là nền tảng để rolling update ở bài Swarm hoạt động an toàn.

5. Registry — kho phân phối image

Registry là nơi *lưu và chia sẻ* image: Docker Hub (công khai), GitLab/GitHub Container Registry, Harbor (tự host). Bạn **push** image lên registry, môi trường khác **pull** về chạy — đây là cách image di chuyển từ CI đến production.

Dockerfile: công thức tạo image

```
FROM node:20-alpine      # image nền – vì sao alpine? nhẹ hơn nhiều
WORKDIR /app             # thư mục làm việc trong container
COPY package*.json ./    # copy file phụ thuộc trước (xem bài Compose để hiểu vì sao)
RUN npm ci               # cài phụ thuộc → tạo một layer
COPY . .                 # copy phần còn lại của source
EXPOSE 3000              # khai báo cổng (mang tính tài liệu)
CMD ["node", "server.js"] # lệnh chạy khi container khởi động
```

Tại sao RUN, COPY mỗi cái là một layer — và điều đó ảnh hưởng gì? Docker cache từng layer. Nếu layer không đổi, lần build sau dùng lại cache thay vì chạy lại. Đó là lý do thứ tự các dòng trong Dockerfile *cực kỳ* quan trọng cho tốc độ build — chủ đề ta sẽ tối ưu ở bài Compose.

Lệnh Docker thường dùng

```
docker run -p 8080:80 nginx      # chạy nginx, map cổng host:container
docker ps                      # liệt kê container đang chạy
docker ps -a                   # cả container đã dừng
docker exec -it <id> sh         # vào shell bên trong container
docker logs <id>               # xem log
docker build -t myapp:1.0 .     # build image từ Dockerfile
docker images                  # liệt kê image
docker pull / push <image>     # kéo về / đẩy lên registry
docker volume ls               # liệt kê volume
docker network ls              # liệt kê network
```

Thực hành: 5 lab Docker mini (xem đáp án ở Bài giải)

1. **Nginx:** `docker run -p 8080:80 nginx` → mở trình duyệt.
2. **Dockerfile:** viết Dockerfile cho một app "Hello World", build & run.
3. **Volume:** mount volume, ghi dữ liệu, xóa container → kiểm chứng dữ liệu còn.
4. **Network:** tạo custom network, chạy 2 container, cho chúng ping nhau bằng tên.
5. **Push:** đẩy image vừa build lên GitLab/GitHub Container Registry.

Đầu ra của bài: Bạn phân biệt được container với VM, giải thích được vì sao volume tách khỏi container, và tự đóng gói được một app thành image đẩy lên registry.

Tiếp theo: [Docker Compose](#) — điều phối nhiều container → [Bài giải: 5 lab Docker](#)

NỘI DUNG · 90'

Docker Compose & Triển khai ứng dụng

Vì sao cần Compose khi có nhiều container, cách Frontend–Backend giao tiếp, và tối ưu image bằng multi-stage build.

Một ứng dụng thật hiếm khi chỉ có một container: frontend, backend, database, cache... Quản lý chúng bằng lệnh `docker run` thủ công nhanh chóng trở thành ác mộng. Đó là lúc Docker Compose vào cuộc.



Tại sao cần Compose: nỗi đau của `docker run` thủ công

Để chạy một app 3 tầng bằng tay, bạn phải: tạo network, chạy DB kèm volume và biến môi trường, chạy backend nối vào network, chạy frontend map cổng, nhớ đúng thứ tự... Mỗi lần khởi động lại là gõ lại hàng chục lệnh, và không ai khác tái tạo được chính xác.

Docker Compose mô tả toàn bộ điều đó trong *một file YAML*. Một lệnh `docker compose up` dựng cả hệ thống. Đây chính là **Infrastructure as Code** áp dụng cho môi trường local.

Tại sao "mô tả bằng file" tốt hơn "gõ lệnh"? File thì version được trong Git, review được, và là *tài liệu sống* luôn đúng với thực tế. Người mới vào đội chỉ cần `git clone` rồi `docker compose up` là có nguyên môi trường — không còn trang wiki "cách dựng máy local" dài 3 trang và luôn lỗi thời.

Giải phẫu `docker-compose.yml`

```
services:
  frontend:
    build: ./frontend           # build từ Dockerfile trong ./frontend
    ports: ["3000:80"]          # chỉ frontend cần lộ ra ngoài
    depends_on: [backend]       # khởi động sau backend
    networks: [app-net]

  backend:
    build: ./backend            # Golang hoặc Node.js
    ports: ["8000:8000"]
    environment:
      DB_HOST: db               # gọi database bằng TÊN service
    networks: [app-net]

  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db-data:/var/lib/postgresql/data  # dữ liệu vào volume
    networks: [app-net]

networks:
  app-net:                      # mạng riêng cho cụm app này

volumes:
  db-data:                     # volume tồn tại độc lập với container
```

Network: vì sao Frontend gọi được Backend

Cả ba service nằm trong `app-net`. Nhờ DNS nội bộ của Docker (đã học ở bài trước), backend gọi database bằng `db:5432`, và frontend gọi backend bằng `backend:8000` — **không cần biết IP**.

Tại sao chỉ frontend cần ports, còn backend và db thì không nhất thiết? ports mở cổng ra thế giới bên ngoài (máy host). Backend và DB chỉ cần được truy cập nội bộ trong network bởi các service khác. Không lộ DB ra ngoài là nguyên tắc bảo mật cơ bản: giảm bề mặt tấn công. Compose mặc định cho phép các service trong cùng network thấy nhau qua cổng nội bộ mà không cần ports.

depends_on và sự thật về “thứ tự khởi động”

depends_on đảm bảo backend khởi động sau db. Nhưng chú ý: nó chỉ đợi container db chạy, không đợi Postgres sẵn sàng nhận kết nối.

Tại sao đây là cái bẫy kinh điển? "Container đang chạy" ≠ "dịch vụ bên trong đã sẵn sàng". Postgres cần vài giây để khởi tạo. Nếu backend kết nối ngay, nó sẽ lỗi. Giải pháp đúng: healthcheck + depends_on: condition: service_healthy, hoặc code backend tự retry kết nối. Bài học sâu hơn: trong hệ phân tán, luôn giả định phụ thuộc có thể chưa sẵn sàng và thiết kế để chịu được điều đó.

Volume để backup dữ liệu

Vì dữ liệu Postgres nằm trong volume db-data, bạn có thể backup mà không đụng tới container:

```
# Backup volume ra file tar
docker run --rm \
  -v db-data:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/db-backup.tar.gz -C /data .
```

Kỹ thuật: chạy một container tạm gắn cả volume dữ liệu lẫn thư mục hiện tại, rồi nén. Container tạm biến mất (--rm) nhưng file backup ở lại.

Từ container thành Image: commit vs build

Bạn có thể tạo image từ một container đang chạy bằng docker commit. Nhưng nên hiểu vì sao đây không phải cách nên dùng thường xuyên:

```
docker commit <container_id> myapp:snapshot # chụp trạng thái container
thành image
```

Tại sao build từ Dockerfile luôn tốt hơn commit ? commit tạo image từ một trạng thái bạn nghịch bằng tay — không ai biết bên trong có gì, không tái tạo được. Dockerfile là *công thức minh bạch, version được*: ai đọc cũng hiểu, ai build cũng ra kết quả giống nhau. commit chỉ hợp để "chụp nhanh" khi debug, không dùng cho production.

Tối ưu Dockerfile: cache layer & multi-stage build

Đây là phần "why" quan trọng nhất của bài. Nhớ ở bài trước: mỗi dòng Dockerfile là một layer, và Docker cache layer không đổi.

Vì sao thứ tự dòng quyết định tốc độ build

```
# ❌ KÉM: copy hết source trước
COPY . .
RUN npm ci          # mỗi lần sửa 1 dòng code → npm ci chạy lại từ đầu!

# ✅ TỐT: copy file phụ thuộc trước
COPY package*.json ./
RUN npm ci          # chỉ chạy lại khi package.json đổi
COPY . .            # sửa code chỉ ảnh hưởng layer này
```

Tại sao đảo hai dòng lại tiết kiệm hàng phút mỗi lần build? npm ci là bước chậm nhất. Nếu nó nằm *sau* COPY . . , thì mỗi lần sửa một dòng code, cache bị vô hiệu và phải cài lại toàn bộ thư viện. Đặt COPY package*.json trước, layer cài đặt chỉ chạy lại khi danh sách thư viện thật sự thay đổi. Nguyên tắc chung: **đặt thứ ít đổi lên trên, thứ hay đổi xuống dưới.**

Multi-stage build (2 phase): image nhỏ và an toàn hơn

```
# Phase 1: build – cần toolchain đầy đủ (nặng)
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # tạo ra thư mục dist/

# Phase 2: runtime – chỉ cần file tĩnh + web server (nhẹ)
FROM nginx:alpine AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

Tại sao tách 2 phase lại quan trọng? Phase build cần Node, npm, mã nguồn, thư viện dev — hàng trăm MB, và chứa cả những thứ có thể là lỗ hổng bảo mật. Nhưng để *chạy* một app React đã build xong, bạn chỉ cần file HTML/JS tĩnh và một web server. Multi-stage vứt bỏ toàn bộ phase build, image cuối chỉ còn nginx + file tĩnh — nhỏ hơn nhiều lần, khởi động nhanh hơn, bề mặt tấn công nhỏ hơn. Image nhỏ còn nghĩa là `pull` nhanh hơn ở mọi node trong cụm Swarm.

Thực hành (xem đáp án ở Bài giải)

1. Viết Dockerfile cho Backend (Go/Node.js) & Frontend (React.js).
2. Thiết lập network để Frontend ↔ Backend giao tiếp.
3. Dùng volume để backup dữ liệu database.
4. Từ container đã chạy, tạo Image mới bằng `commit` (và so sánh với build).
5. `docker compose up` toàn bộ stack, kiểm tra end-to-end.

Đầu ra của bài: Bạn dựng được một app đa container bằng Compose, giải thích được vì sao network/volume tách bạch, và tối ưu được Dockerfile bằng cache layer + multi-stage.

Tiếp theo: [Docker Swarm — vận hành cụm container](#) → · **Bài giải:** [Compose & triển khai](#)

NỘI DUNG · 75'

Docker Cluster & Docker Swarm

Vì sao một host là không đủ, orchestration giải quyết gì, và cách dựng cụm Swarm với self-healing & rolling update.

Compose tuyệt vời cho *một máy*. Nhưng production thật cần nhiều máy: để chịu tải, để không sập khi một máy chết. Bước từ "một host" lên "cụm nhiều host" là lúc bạn cần **orchestration** — và Docker Swarm là điểm khởi đầu dễ nhất.



Tại sao một host là không đủ

Chạy toàn bộ app trên một server duy nhất có ba giới hạn chí mạng:

1. **Không chịu lỗi (no fault tolerance):** máy đó chết → toàn bộ app chết.
2. **Không scale ngang:** hết CPU/RAM thì chỉ còn cách nâng cấp máy (scale up) — đắt và có trần.
3. **Deploy gây gián đoạn:** cập nhật app phải tắt container cũ → người dùng thấy downtime.

Tại sao "scale ngang" (thêm máy) lại đáng mơ ước hơn "scale dọc" (máy to hơn)? Máy to hơn có giới hạn vật lý và giá tăng phi tuyến. Thêm nhiều máy thường rẻ hơn, và quan trọng hơn: nhiều máy cho bạn *dự phòng* — một máy chết, các máy khác gánh tiếp. Orchestration sinh ra để biến "nhiều máy rời rạc" thành "một cụm hoạt động như một khối".

Orchestration giải quyết gì?

Một công cụ orchestration (điều phối) lo giúp bạn những việc mà làm tay là bất khả thi ở quy mô lớn:

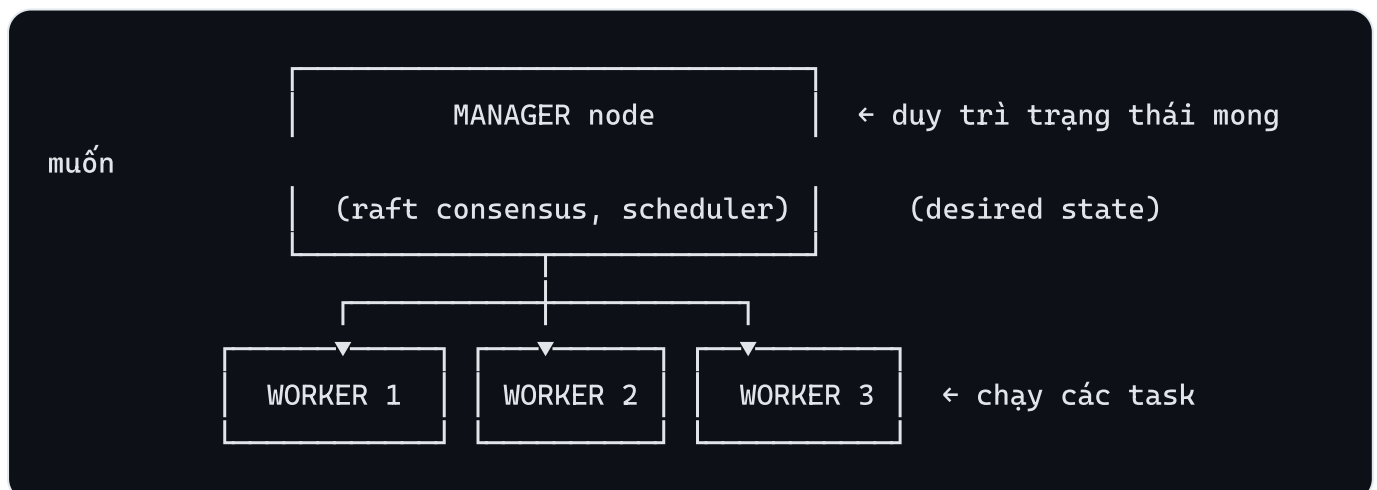
- **Scheduling:** đặt container nào lên máy nào cho cân tải.
- **Self-healing:** container chết → tự khởi động lại; máy chết → tự dời container sang máy khác.
- **Scaling:** tăng/giảm số bản sao theo lệnh (hoặc tự động).
- **Rolling update:** cập nhật từng phần, không downtime.
- **Service discovery & load balancing:** phân phối request tới các bản sao.

Chọn công cụ nào: Swarm, Kubernetes hay Nomad?

	Docker Swarm	Kubernetes	Nomad
Độ phức tạp	Thấp — tích hợp sẵn trong Docker	Cao — nhiều khái niệm	Trung bình
Học nhanh	✅ Rất nhanh	❌ Dốc	Vừa
Hệ sinh thái	Nhỏ	Khổng lồ, chuẩn công nghiệp	Đang lớn
Hợp cho	Học nền tảng, cụm nhỏ/vừa	Production quy mô lớn	Workload hỗn hợp

Tại sao học Swarm trước Kubernetes, dù K8s mới là "chuẩn"? Vì Swarm dạy bạn *khái niệm* orchestration (service, replica, overlay network, rolling update) với chi phí nhận thức tối thiểu — nó dùng chính cú pháp Compose bạn đã biết. Nắm vững "tại sao" ở Swarm rồi, bạn học Kubernetes như học một triển khai mạnh hơn của cùng những ý tưởng đó, thay vì học từ số không giữa một rừng thuật ngữ.

Kiến trúc Swarm: Manager và Worker



- **Manager node:** giữ *trạng thái mong muốn* của cụm ("tôi muốn 3 bản sao của web"), lập lịch, và phối hợp qua thuật toán **Raft**.
- **Worker node:** thực thi các **task** (mỗi task là một container) mà manager giao.

Tại sao Swarm xoay quanh khái niệm "desired state"? Bạn không ra lệnh "chạy container này trên máy kia". Bạn *khai báo* "tôi muốn luôn có 3 bản sao web đang chạy". Manager liên tục so sánh thực tế với mong muốn và tự sửa cho khớp. Đây là khác biệt cốt lõi giữa *imperative* (ra lệnh từng bước) và *declarative* (khai báo kết quả) — và là lý do self-healing hoạt động: khi một bản sao chết, thực tế lệch khỏi mong muốn, manager tự tạo bản mới.

Các khái niệm Swarm

- **Service:** định nghĩa "chạy image X với N bản sao" — đơn vị bạn thao tác, thay cho container lẻ.
- **Task:** một bản sao (container) cụ thể của service.
- **Replica:** số bản sao mong muốn của một service.
- **Overlay network:** mạng ảo trải *nhiều host*, để container ở các máy khác nhau vẫn nói chuyện được.
- **Ingress routing mesh:** gửi request tới *bất kỳ* node nào trong cụm, Swarm tự định tuyến tới bản sao còn sống.

Tại sao cần overlay network, trong khi Compose chỉ cần bridge? Bridge network chỉ sống trong một host. Khi container nằm rải trên nhiều máy vật lý khác nhau, chúng cần một mạng ảo "phủ lên trên" hạ tầng mạng thật — đó là overlay. Nó cho phép `backend` ở máy 1 gọi `db` ở máy 3 bằng tên, y như khi chúng ở cùng một máy.

Dựng và vận hành cụm

```
# 1) Khởi tạo cụm trên MANAGER
docker swarm init --advertise-addr <IP_MANAGER>

# 2) JOIN worker (chạy lệnh manager in ra)
docker swarm join --token <token> <IP_MANAGER>:2377

# 3) Kiểm tra các node
docker node ls

# 4) Deploy service với 3 bản sao
docker service create --name web --replicas 3 -p 80:80 nginx
docker service ls
docker service ps web          # xem các task nằm ở node nào

# 5) Scale lên 5 bản sao – chỉ một lệnh
docker service scale web=5

# 6) Rolling update sang image mới, không downtime
docker service update --image nginx:alpine web

# 7) Deploy cả stack từ file Compose
docker stack deploy -c docker-compose.yml myapp
```

Tại sao rolling update không gây downtime? Swarm cập nhật *từng bản sao một*: tắt bản 1, dựng bản 1 phiên bản mới, đợi nó khỏe, rồi mới sang bản 2... Trong suốt quá trình luôn có bản sao đang phục vụ, nên người dùng không thấy gián đoạn. Nếu bản mới lỗi, bạn `docker service rollback` để quay về phiên bản cũ. Đây chính là lý do ta cần *nhiều* bản sao ngay từ đầu — không chỉ để chịu tải, mà để cập nhật an toàn.

Self-healing: thử nghiệm đáng nhớ nhất

Trong lab, hãy tắt một worker node và quan sát. Manager phát hiện các task trên node đó “chết”, so với desired state thấy thiếu bản sao, và **tự tạo lại** trên các node còn sống. Bạn không làm gì cả — cụm tự lành.

Tại sao đây là đỉnh cao của cả khóa học? Nhìn lại hành trình: Git cho ta lịch sử tái lập → CI/CD tự động hóa build/test → Docker đóng gói môi trường bất biến → Compose mô tả hệ thống bằng file → Swarm biến mô tả đó thành một cụm tự vận hành, tự lành. Mỗi mắt xích giải quyết một "tại sao" cụ thể, và ráp lại thành thứ mà trước DevOps là bất khả thi: deploy nhiều lần mỗi ngày, an toàn, không cần thức đêm.

Thực hành (xem đáp án ở Bài giải)

1. `docker swarm init` trên manager, join 2 worker.
2. Deploy service nginx 3 bản sao, kiểm tra phân bố task.
3. Scale lên 5, rồi rolling update sang image khác.
4. `docker stack deploy` toàn bộ app full-stack từ bài trước.
5. Tắt một node → quan sát self-healing.

Đầu ra của bài: Bạn dựng được cụm Swarm nhiều node, giải thích được desired state & self-healing, và deploy được một stack hoàn chỉnh lên cụm.

Ôn lại toàn bộ: [Về trang chủ](#) · **Bài giải:** [Docker Swarm](#) · **Mở rộng:** [Reference](#) — sách & video

PHẦN II

Bài giải

BÀI GIẢI

Bài giải: Git & GitLab

Lời giải cho phần thực hành quản lý mã nguồn: nhánh, Merge Request và xử lý xung đột.

Bài tập gốc ở [Nội dung · Version Control](#). Hãy tự làm trước khi xem đáp án.

1. Tạo repo và clone về máy

Sau khi bấm **New project** → **Create blank project** trên GitLab:

```
git clone https://gitlab.com/<user>/<repo>.git
cd <repo>
git config user.name "Ten Ban"
git config user.email "email@cua-ban.com"
```

2. Nhánh feature, commit, push

```
git switch -c feature/login      # tạo & chuyển sang nhánh mới
# ... sửa code, ví dụ tạo file login.js ...
git add login.js
git commit -m "Thêm màn hình đăng nhập"

Vì sao: tách logic auth khỏi trang chủ để dễ test độc lập."
git push -u origin feature/login # -u để lần sau chỉ cần 'git push'
```

Vì sao commit message có phần "Vì sao"? Dòng đầu tóm tắt *cái gì*; phần thân giải thích *tại sao*. 6 tháng sau, phần "tại sao" là thứ cứu bạn.

3. Merge Request

1. Push xong, GitLab hiện link tạo MR — hoặc vào **Merge requests** → **New**.
2. Chọn source `feature/login` → target `main`.

3. Gán reviewer, mô tả thay đổi.
4. Reviewer duyệt → **Merge**. Bật *Delete source branch* để dọn nhánh đã xong.

4. Tạo & giải quyết xung đột

Dự tình huống: hai nhánh cùng sửa một dòng.

```
# Nhánh A
git switch -c feature/a main
# sửa dòng 1 của README.md thành "Xin chào A"
git commit -am "A sửa dòng 1"

# Nhánh B
git switch -c feature/b main
# sửa đúng dòng 1 thành "Xin chào B"
git commit -am "B sửa dòng 1"

# Merge A vào main trước
git switch main && git merge feature/a
# Merge B → XUNG ĐỘT
git merge feature/b
```

Git đánh dấu xung đột trong file:

```
<<<<<< HEAD
Xin chào A
=====
Xin chào B
>>>>>> feature/b
```

Sửa tay để giữ nội dung đúng (xóa các dấu <<<, ≡, >>>), rồi:

```
git add README.md
git commit          # hoàn tất merge
```

Vì sao Git bắt *con người* giải quyết xung đột? Vì chỉ con người mới biết ý định. Khi hai thay đổi mâu thuẫn về ngữ nghĩa, không có quy tắc máy móc nào chọn đúng được — Git dừng lại và hỏi bạn thay vì đoán mò và làm hỏng code.

Nội dung liên quan: [Version Control](#)

BÀI GIẢI

Bài giải: Dựng pipeline CI/CD

Lời giải: register GitLab Runner (kèm batch/shell tự động), viết pipeline, và làm GitHub Actions tương đương.

Bài tập gốc ở [Nội dung · CI/CD](#).

1. Register GitLab Runner (executor Docker)

Cách nhanh nhất là chạy runner trong... một container Docker:

```
# Chạy runner service
docker run -d --name gitlab-runner --restart always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v gitlab-runner-config:/etc/gitlab-runner \
  gitlab/gitlab-runner:latest

# Register (lấy token ở Settings → CI/CD → Runners của repo)
docker run --rm -it \
  -v gitlab-runner-config:/etc/gitlab-runner \
  gitlab/gitlab-runner register \
  --non-interactive \
  --url "https://gitlab.com/" \
  --token "<REGISTRATION_TOKEN>" \
  --executor "docker" \
  --docker-image "docker:24" \
  --description "docker-runner"
```

AI-assisted: script batch dựng runner (Windows)

Đây là loại việc lặp lại, dễ sai — hợp để nhờ AI sinh script rồi bạn đọc hiểu:

```

@echo off
REM setup-runner.bat - dựng GitLab Runner trên Windows (Docker Desktop)
SET TOKEN=%1
IF "%TOKEN%"==" " (
    echo Cách dùng: setup-runner.bat ^<REGISTRATION_TOKEN^>
    exit /b 1
)

docker run -d --name gitlab-runner --restart always ^
-v //var/run/docker.sock:/var/run/docker.sock ^
-v gitlab-runner-config:/etc/gitlab-runner ^
gitlab/gitlab-runner:latest

docker run --rm ^
-v gitlab-runner-config:/etc/gitlab-runner ^
gitlab/gitlab-runner register --non-interactive ^
--url "https://gitlab.com/" ^
--token "%TOKEN%" ^
--executor "docker" ^
--docker-image "docker:24" ^
--description "win-docker-runner"

echo Xong. Kiểm tra ở Settings ^> CI/CD ^> Runners.

```

Vì sao mount `docker.sock` vào runner? Để job bên trong runner có thể ra lệnh cho Docker của host — cần thiết khi pipeline phải `docker build`. Đây cũng là điểm cần lưu ý về bảo mật: ai điều khiển được socket này gần như điều khiển được host.

2. Pipeline `.gitlab-ci.yml`

```
stages: [build, test]

build:
  stage: build
  image: node:20
  script:
    - npm ci
    - npm run build
  artifacts:
    paths: [dist/]
    expire_in: 1 week

test:
  stage: test
  image: node:20
  script:
    - npm test
```

Push file này lên repo → vào **Build** → **Pipelines** để xem chạy, đọc log từng job, và tải artifact `dist/`.

3. GitHub Actions tương đương

```
# .github/workflows/ci.yml
name: CI
on: [push]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run build
      - run: npm test
      - uses: actions/upload-artifact@v4
        with: { name: dist, path: dist/ }
```

So sánh

	GitLab CI	GitHub Actions
Cài môi trường	<code>image: node:20</code>	<code>setup-node</code> action
Artifact	<code>artifacts:</code>	<code>upload-artifact</code> action
Cảm giác	Khai báo gọn, ít "lắp ghép"	Ghép action như lego, marketplace giàu

Nội dung liên quan: [CI/CD](#)

BÀI GIẢI

Bài giải: 5 lab Docker mini

Lời giải chi tiết cho 5 lab: nginx, Dockerfile, volume, network và push image lên registry.

Bài tập gốc ở [Nội dung · Docker Fundamentals](#).

Lab 1 — Chạy nginx

```
docker run -d --name web -p 8080:80 nginx
# Mở http://localhost:8080
docker logs web      # xem log truy cập
docker rm -f web     # dọn dẹp
```

`-p 8080:80` nghĩa là cổng 8080 của host → cổng 80 trong container.

Lab 2 — Dockerfile “Hello World”

`server.js` :

```
require('http')
  .createServer((_, res) => res.end('Hello from Docker!'))
  .listen(3000);
```

Dockerfile :

```
FROM node:20-alpine
WORKDIR /app
COPY server.js .
EXPOSE 3000
CMD ["node", "server.js"]
```



```
docker build -t hello:1.0 .
docker run -d -p 3000:3000 hello:1.0
# http://localhost:3000 → Hello from Docker!
```

Lab 3 — Volume giữ dữ liệu qua vòng đời container

```
docker volume create mydata
# Ghi dữ liệu vào volume
docker run --rm -v mydata:/data alpine sh -c "echo 'du lieu quan trong' > /data/note.txt"
# Xóa mọi container... rồi đọc lại từ container MỚI
docker run --rm -v mydata:/data alpine cat /data/note.txt
# → du lieu quan trong (dữ liệu vẫn còn!)
```

Vì sao dữ liệu sống sót? Vì nó nằm trong volume `mydata` do Docker quản lý, hoàn toàn tách khỏi container. Container chỉ là tiến trình phù du; volume mới là nơi trạng thái tồn tại.

Lab 4 — Hai container ping nhau bằng tên

```
docker network create appnet

docker run -d --name api --network appnet nginx
docker run --rm --network appnet alpine sh -c "apk add -q curl && curl -s api"
# → trang mặc định của nginx (phân giải 'api' qua DNS nội bộ)
```

Nếu *không* đặt cùng network, `curl api` sẽ báo “could not resolve host” — chứng minh vì sao custom network là bắt buộc để service discovery hoạt động.

Lab 5 — Push image lên registry

Ví dụ với GitHub Container Registry (GHCR):

```
# Đăng nhập bằng Personal Access Token (scope: write:packages)
echo $CR_PAT | docker login ghcr.io -u <username> --password-stdin

docker tag hello:1.0 ghcr.io/<username>/hello:1.0
docker push ghcr.io/<username>/hello:1.0
```

Với GitLab Container Registry:

```
docker login registry.gitlab.com
docker tag hello:1.0 registry.gitlab.com/<user>/<repo>/hello:1.0
docker push registry.gitlab.com/<user>/<repo>/hello:1.0
```

Vì sao phải tag theo đúng địa chỉ registry trước khi push? Tên image mã hóa cả đích đến. `ghcr.io/user/hello` nói cho Docker biết đẩy đi đâu. Không có tiền tố registry, Docker mặc định nhắm Docker Hub.

Nội dung liên quan: [Docker Fundamentals](#)

BÀI GIẢI

Bài giải: Compose & triển khai full-stack

Lời giải: Dockerfile backend/frontend, network, backup volume, và Dockerfile multi-stage tối ưu.

Bài tập gốc ở [Nội dung · Docker Compose](#).

1. Dockerfile

Backend (Node.js):

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 8000
CMD ["node", "server.js"]
```

Backend (Go) — multi-stage, binary tĩnh:

```
FROM golang:1.22 AS build
WORKDIR /src
COPY go.* ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app/server .

FROM alpine:3.20
COPY --from=build /app/server /server
EXPOSE 8000
CMD ["/server"]
```

Frontend (React) — multi-stage:

```
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

Vì sao Go build ra image alpine chỉ vài MB? `CGO_ENABLED=0` tạo một binary tĩnh không phụ thuộc thư viện hệ thống. Image cuối chỉ cần chính binary đó — không cần cả toolchain Go. Đây là multi-stage build đẩy tới cực hạn.

2. `docker-compose.yml` + network

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:80"]
    depends_on: [backend]
    networks: [app-net]

  backend:
    build: ./backend
    environment:
      DB_HOST: db
      DB_PASSWORD: secret
    networks: [app-net]

  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes: [db-data:/var/lib/postgresql/data]
    networks: [app-net]

networks: { app-net: {} }
volumes: { db-data: {} }
```

```
docker compose up -d --build
docker compose ps
docker compose logs -f backend
```

Frontend gọi `http://backend:8000` , backend gọi `db:5432` — tất cả qua tên service.

3. Backup volume database

```
docker run --rm \
  -v db-data:/data \
  -v "$(pwd)":/backup \
  alpine tar czf /backup/db-$(date +%F).tar.gz -C /data .
```

Khôi phục:

```
docker run --rm -v db-data:/data -v "$(pwd)":/backup \
  alpine sh -c "cd /data && tar xzf /backup/db-2026-07-11.tar.gz"
```

4. Từ container thành image

```
docker compose exec backend sh # vào chỉnh gì đó (chỉ để minh họa)
docker commit <backend_container_id> mybackend:snapshot
docker images | grep mybackend
```

So sánh: image từ `commit` chạy được nhưng "hộp đen"; image từ Dockerfile minh bạch và tái lập.

Production luôn dùng Dockerfile.

5. Chạy end-to-end

```
docker compose up -d --build
# Mở http://localhost:3000 - frontend gọi backend, backend đọc/ghi db
docker compose down # dừng (giữ volume)
docker compose down -v # dừng và XÓA cả volume - cẩn thận!
```

Nội dung liên quan: [Docker Compose](#)

BÀI GIẢI

Bài giải: Docker Swarm

Lời giải: khởi tạo cụm, deploy service, scale, rolling update, stack deploy và thử self-healing.

Bài tập gốc ở [Nội dung · Docker Swarm](#).

1. Khởi tạo cụm & join worker

Trên **manager**:

```
docker swarm init --advertise-addr 192.168.1.10
# In ra một lệnh 'docker swarm join --token SWMTKN-... 192.168.1.10:2377'
```

Trên mỗi **worker**, dán lệnh vừa in. Nếu lỡ mất token:

```
docker swarm join-token worker      # in lại lệnh join cho worker
```

Kiểm tra trên manager:

```
docker node ls
# ID      HOSTNAME      STATUS      AVAILABILITY      MANAGER STATUS
# ...    manager      Ready       Active             Leader
# ...    worker1      Ready       Active
# ...    worker2      Ready       Active
```

2. Deploy service 3 bản sao

```
docker service create --name web --replicas 3 -p 80:80 nginx
docker service ls
docker service ps web      # thấy 3 task nằm rải trên các node
```

Nhờ **ingress routing mesh**, gọi cổng 80 tới bất kỳ IP node nào cũng tới được web.

3. Scale & rolling update

```
# Scale 3 → 5
docker service scale web=5
docker service ps web          # 5 task

# Rolling update sang image mới, kiểm soát nhịp
docker service update \
  --image nginx:alpine \
  --update-parallelism 1 \
  --update-delay 10s \
  web
```

Vì sao đặt `--update-parallelism 1` và `--update-delay` ? Để cập nhật *từng* bản sao một, cách nhau 10 giây. Nếu bản mới lỗi, bạn phát hiện sớm khi mới hỏng 1 bản, thay vì cập nhật đồng loạt rồi sập cả 5. Đây là "phanh an toàn" cho deploy.

Rollback khi cần:

```
docker service rollback web
```

4. Deploy stack từ Compose

Swarm dùng lại chính file Compose (phiên bản có `deploy:`):

```
# stack.yml
services:
  web:
    image: nginx:alpine
    ports: ["80:80"]
    deploy:
      replicas: 3
      restart_policy: { condition: on-failure }
      update_config: { parallelism: 1, delay: 10s }
```



```
docker stack deploy -c stack.yml myapp
docker stack services myapp
docker stack ps myapp
```

Vì sao khối `deploy`: bị Compose thường bỏ qua nhưng Swarm mới đọc? Vì `replicas`, `restart_policy`, `update_config` là khái niệm *cụm* — chỉ có nghĩa khi có orchestrator. Cùng một file mô tả được cả môi trường local (Compose) lẫn production (Swarm); mỗi bên đọc phần liên quan tới mình.

5. Thử self-healing

```
# Trên worker1: mô phỏng node chết
sudo systemctl stop docker          # hoặc tắt máy ảo

# Trên manager: quan sát
watch docker service ps web
# Các task ở worker1 chuyển 'Shutdown', task MỖI được tạo trên node còn sống
```

Manager thấy thực tế (số task sống) lệch khỏi desired state (5 bản sao) và tự tạo lại — không cần bạn can thiệp. Bật lại worker1, nó gia nhập cụm trở lại và sẵn sàng nhận task mới.

Nội dung liên quan: [Docker Swarm](#) · Kết thúc khóa — xem [Reference](#) để học sâu hơn.

PHỤ LỤC

Tham khảo

THAM KHẢO

Reference — Sách & Video

Nơi tổng hợp tài liệu mở rộng. Thêm link ebook và video của bạn vào từng mục bên dưới.

Trang này là nơi bạn **tự thêm** link ebook và video cho từng chủ đề. Chỉ cần sửa file

`src/pages/reference.md` — thêm một dòng `- [Tên tài liệu](URL)` vào đúng mục. Cấu trúc dưới đây đã bám theo 5 session của khóa học.

 **Quy ước:** Mỗi mục có hai nhóm —  **Ebook / bài viết** và  **Video**. Xóa dòng `_(chưa có - thêm link tại đây)_` khi bạn đã thêm.

0. Tư duy DevOps

Ebook / bài viết

- [The Phoenix Project](#) — tiểu thuyết kinh điển về DevOps
- [The DevOps Handbook](#)
- *(chưa có — thêm link tại đây)*

Video

- *(chưa có — thêm link tại đây)*

1. Version Control (Git, GitHub, GitLab)

Ebook / bài viết

- [Pro Git](#) (sách miễn phí, có bản tiếng Việt)
- [GitLab Docs](#) — Install
- *(chưa có — thêm link tại đây)*

Video

- *(chưa có — thêm link tại đây)*

2. CI/CD (GitLab Runner, GitHub Actions)

Ebook / bài viết

- [GitLab CI/CD Documentation](#)
- [GitHub Actions Documentation](#)
- *(chưa có — thêm link tại đây)*

Video

- *(chưa có — thêm link tại đây)*

3. Docker Fundamentals

Ebook / bài viết

- [Docker — Get Started](#)
- [Dockerfile best practices](#)
- *(chưa có — thêm link tại đây)*

Video

- *(chưa có — thêm link tại đây)*

4. Docker Compose & Triển khai

Ebook / bài viết

- [Docker Compose Documentation](#)
- [Multi-stage builds](#)
- *(chưa có — thêm link tại đây)*

Video

- *(chưa có — thêm link tại đây)*

5. Docker Cluster & Swarm

Ebook / bài viết

- [Swarm mode overview](#)

- **Deploy a stack to a swarm**
- *(chưa có — thêm link tại đây)*

Video

- *(chưa có — thêm link tại đây)*

Cách thêm một mục hoàn toàn mới

Muốn thêm một chủ đề ngoài 5 session? Copy khối mẫu này vào cuối file:

```
## <Tên chủ đề mới>

**  Ebook / bài viết**
- [Tên tài liệu](https://link-cua-ban)

**  Video**
- [Tên video](https://link-video)
```